

Relatório Técnico do Grupo Servidor

Dinâmica IntegraLab, cenário ConectaShop · Equipe S13

1. Identificação da equipe

Este relatório apresenta a implementação dos servidores REST e gRPC da equipe S13 na dinâmica IntegraLab (cenário ConectaShop) e o resultado da integração com o Cliente sorteado.

Código da equipe	S13
Disciplina	Sistemas Distribuídos e Computação Paralela
Professor	Wolney
Data da integração	06/10/2026
Cliente sorteado	C5 (REST e gRPC)
Repositório	github.com/GSoaresTech/conectashop-marketplace

Integrante	Responsabilidade
Gabriel Soares	Servidor REST (Catalog & Quote API)
Gabriel Marques	Servidor gRPC (ShippingService)
Deyvid Gustavo	Relatório técnico e testes de validação

2. Tecnologias escolhidas

A dinâmica deixava a tecnologia livre, desde que os contratos fossem seguidos à risca. A equipe escolheu as tecnologias abaixo.

Componente	Linguagem	Bibliotecas	Porta padrão
Servidor REST	Python 3.13	FastAPI 0.142, Uvicorn 0.54, Pydantic 2.13	8080
Servidor gRPC	Python 3.13	grpcio 1.84, grpcio-tools 1.84 (protobuf)	50051
Testes	Python 3.13	pytest 9.1 (27 testes automatizados do REST), testes_grpc.py (G1 a G5) e um script de validação com requests e stubs grpcio	-

Motivo da escolha: a equipe já tinha familiaridade com Python. O FastAPI permite declarar o corpo da cotação com Pydantic e centralizar o tratamento de erros, e o grpcio é a implementação oficial de gRPC para Python, com geração de stubs a partir do .proto pelo grpcio-tools.

3. Arquitetura dos servidores

Os dois servidores são processos independentes e não usam banco de dados: o catálogo e as tabelas de frete ficam em memória. Ambos fazem bind em 0.0.0.0 para aceitar conexões de outras máquinas da rede do laboratório. O código da equipe e as portas vêm de variáveis de ambiente (SERVER_TEAM, REST_PORT e GRPC_PORT).

Serviço	Ponto de entrada	Endereço	Log
REST	rest-server/app/main.py	0.0.0.0:8080, base path /api/v1	logs/rest/server.log
gRPC	grpc-server/main.py	0.0.0.0:50051	logs/grpc/server.log

3.1 Servidor REST (Catalog & Quote API)

Expõe dois endpoints sob o base path /api/v1:

- GET /api/v1/products/{sku} consulta um produto do catálogo fixo, com 4 SKUs.
- POST /api/v1/quotes valida os itens e calcula subtotal, desconto (0%, 5% ou 10%) e total, em centavos inteiros.

Fluxo de uma requisição:

1. Validação dos headers X-Client-Team e X-Request-ID. Se faltar algum, a resposta é 400 MISSING_REQUIRED_HEADER.
2. Validação do corpo e das regras de negócio, com 400, 404 ou 422 conforme o contrato.
3. Cálculo e resposta em JSON, devolvendo X-Request-ID no header.
4. Registro da chamada no log.

Organização do código (rest-server/app/): main.py cria a aplicação, as rotas, a validação dos headers e o middleware que devolve X-Request-ID e registra o log; catalog.py guarda o catálogo fixo; quotes.py tem o modelo do pedido (Pydantic, com quantity como inteiro estrito), as validações e o cálculo; errors.py define o formato padrão de erro e converte erros de corpo em 400 INVALID_REQUEST; logs.py grava as linhas de log; config.py lê SERVER_TEAM, REST_HOST e REST_PORT. Os testes ficam em rest-server/tests/test_api.py.

3.2 Servidor gRPC (ShippingService)

Usa os stubs gerados a partir do shipping.proto do contrato, sem alterações (package shipping.v1). Implementa dois métodos:

- Health retorna status SERVING e o server_team da equipe.
- CalculateShipping valida a metadata e os campos e depois calcula preço e prazo.

Cálculo do frete: quilogramas cobrados = teto(weight_grams / 1000); price_cents = tarifa base (zona × modo) + quilogramas cobrados × adicional (400 no STANDARD, 600 no EXPRESS). O prazo vem da tabela zona × modo.

Erros de validação retornam INVALID_ARGUMENT com as descrições MISSING_CLIENT_TEAM, MISSING_REQUEST_ID, INVALID_WEIGHT, INVALID_ZONE ou INVALID_MODE. Qualquer exceção inesperada vira INTERNAL com INTERNAL_ERROR.

Organização do código (grpc-server/): shipping_pb2.py e shipping_pb2_grpc.py são os stubs gerados pelo grpcio-tools a partir de contracts/grpc/shipping.proto; main.py contém a classe ShippingService (validações, tabelas de tarifa base, adicional por kg e prazo, logs) e a função serve(), que sobe o servidor com um pool de 10 threads em 0.0.0.0; testes_grpc.py é um cliente de apoio que executa G1 a G5 contra o endereço definido em GRPC_TARGET.

3.3 Logs

Cada chamada recebida gera uma linha no formato mínimo pedido no guia, com horário, protocolo, equipe Servidor, equipe Cliente, Request ID, operação, entrada, status e resultado.

Chamadas com erro também são registradas. O primeiro exemplo abaixo vem do log da integração; o segundo, de um teste interno com testes_grpc.py.

```
[2026-10-06T20:32:35] protocol=REST server=S13 client=S13 requestId=req-001 operation=GET_PRODUCT input=KB-100 status=200 result=OK
[2026-10-06 21:44:49,847] protocol=GRPC server=S13 client=C01 requestId=g2-001 operation=CalculateShipping weight=1500 zone=LOCAL mode=STANDARD status=OK priceCents=1800 estimatedDays=2
```

4. Endereços usados na atividade

Código da equipe	S13
REST_BASE_URL	http://172.16.17.59:8080 (base path /api/v1)
GRPC_TARGET	172.16.17.59:50051
Estado no sorteio	REST=UP; gRPC=UP

Os dois serviços rodaram na mesma máquina do laboratório. O acesso ao REST a partir de outra máquina foi confirmado pelas próprias chamadas do C5. O gRPC ficou ativo durante a janela, e o log registra as inicializações com server_team=S13, mas não recebeu chamadas externas. Por isso não há evidência de acesso ao gRPC por outra máquina no laboratório.

5. Testes internos antes do sorteio

Antes da integração, a equipe executou os testes obrigatórios e casos extras de erro com um script cliente em Python (requests para o REST e stubs grpcio gerados do shipping.proto), por localhost e pelo IP da máquina na rede, confirmando o bind em 0.0.0.0 e os valores do contrato. Foram 39 verificações, todas de acordo com o contrato. A suíte pytest do REST, com 27 testes, também passou.

5.1 Testes obrigatórios

Teste	Entrada	Esperado	Obtido	Situação
R1	GET KB-100	200; unitPriceCents=25990	200; unitPriceCents=25990; X-Request-ID devolvido	OK
R2	GET XX-999	404; PRODUCT_NOT_FOUND	404; PRODUCT_NOT_FOUND; requestId=r2	OK
R3	2×KB-100 + 1×MS-200	200; subtotal 64970; 5%; desconto 3248; total 61722	200; 64970; 5%; 3248; 61722	OK
R4	1×MN-400	200; subtotal 119990; 10%; desconto 11999; total 107991	200; 119990; 10%; 11999; 107991	OK
R5	SKU inexistente	422; INVALID_PRODUCT	422; INVALID_PRODUCT	OK
G1	Health()	SERVING; server_team=S13	SERVING; server_team configurado	OK
G2	1500 g, LOCAL, STANDARD	price_cents=1800; 2 dias	price_cents=1800; 2 dias; request_id repetido	OK
G3	2500 g, REGIONAL, EXPRESS	price_cents=4600; 2 dias	price_cents=4600; 2 dias	OK
G4	1000 g, NATIONAL, STANDARD	price_cents=3400; 7 dias	price_cents=3400; 7 dias	OK

Teste	Entrada	Esperado	Obtido	Situação
G5	weight_grams=0	INVALID_ARGUMENT; INVALID_WEIGHT	INVALID_ARGUMENT; INVALID_WEIGHT	OK

5.2 Testes extras de erro

Caso	Esperado	Obtido	Situação
REST sem X-Client-Team	400; MISSING_REQUIRED_HEADER	400; MISSING_REQUIRED_HEADER	OK
REST sem X-Request-ID	400; MISSING_REQUIRED_HEADER	400; MISSING_REQUIRED_HEADER	OK
POST com JSON inválido	400; INVALID_REQUEST	400; INVALID_REQUEST	OK
POST com SKU duplicado	422; INVALID_QUANTITY_OR_ITEMS	422; INVALID_QUANTITY_OR_ITEMS	OK
POST com quantity 0 ou 11	422; INVALID_QUANTITY_OR_ITEMS	422; INVALID_QUANTITY_OR_ITEMS nos dois casos	OK
POST com 0 ou 6 itens	422; INVALID_QUANTITY_OR_ITEMS	422; INVALID_QUANTITY_OR_ITEMS nos dois casos	OK
gRPC sem x-client-team	INVALID_ARGUMENT; MISSING_CLIENT_TEAM	INVALID_ARGUMENT; MISSING_CLIENT_TEAM em Health e CalculateShipping	OK
gRPC com request_id vazio	INVALID_ARGUMENT; MISSING_REQUEST_ID	INVALID_ARGUMENT; MISSING_REQUEST_ID	OK
gRPC com weight_grams=30001	INVALID_ARGUMENT; INVALID_WEIGHT	INVALID_ARGUMENT; INVALID_WEIGHT	OK
gRPC com zone UNSPECIFIED	INVALID_ARGUMENT; INVALID_ZONE	INVALID_ARGUMENT; INVALID_ZONE	OK
gRPC com mode UNSPECIFIED	INVALID_ARGUMENT; INVALID_MODE	INVALID_ARGUMENT; INVALID_MODE	OK

Problemas encontrados nesta fase: nenhuma divergência de contrato. Também foram conferidos limites (quantity=10, peso de 1 g, 1001 g e 30000 g, peso negativo, quantity enviada como texto) e a repetição exata de request_id com acentos e espaços, todos corretos.

Ficaram dois pontos de atenção para a aula. O código padrão da equipe era diferente entre os servidores (S00 no REST e S01 no gRPC), então SERVER_TEAM precisava ser definida com S13 antes de iniciar os dois. E o Cliente gRPC precisa enviar a chave x-client-team em minúsculas, porque o HTTP/2 rejeita chaves de metadata com maiúsculas antes de a chamada sair do Cliente.

6. Matriz de execução

O sorteio atribuiu à equipe S13 um único Cliente, o C5, para os dois protocolos. O guia prevê dois Clientes REST e dois Clientes gRPC por Servidor, então a matriz tem apenas duas linhas.

A aplicação Cliente do C5 apresentou problemas técnicos e não executou a bateria de testes. Para aproveitar a janela, o C5 testou o REST pela página /docs (Swagger UI) que o FastAPI gera no nosso servidor. Nenhuma chamada gRPC chegou.

Protocolo	Cliente	Testes recebidos	Resultado inicial	Resultado final	Request IDs	Observação
REST	C5	R1 e R5, além de casos de erro	A aplicação do C5 não executou os testes	R1 e R5 OK; R2, R3 e R4 não executados	req-001, MN-400	Chamadas feitas pela página /docs do servidor, com X-Client-Team=S13
gRPC	C5	Nenhum	Nenhuma chamada recebida	G1 a G5 não executados	-	Servidor ativo durante a janela

7. Resultados por teste

7.1 REST

Teste	Esperado	Cliente C5	Evidência no log
R1	200; unitPriceCents=25990	OK	20:32:35, req-001
R2	404; PRODUCT_NOT_FOUND	Não executado	-
R3	200; subtotal 64970; desconto 5%; total 61722	Não executado	-
R4	200; subtotal 119990; desconto 10%; total 107991	Não executado	-
R5	422; INVALID_PRODUCT	OK, com o corpo de exemplo do Swagger (SKU string)	20:39:32, req-001

Na chamada das 20:39:32, o corpo de exemplo do Swagger tinha SKU inexistente e quantity 0 ao mesmo tempo. O contrato não define qual erro prevalece nesse caso. O servidor valida o SKU antes da quantidade de cada item e devolveu 422 INVALID_PRODUCT, o resultado esperado em R5.

7.2 gRPC

Teste	Esperado	Cliente C5	Evidência no log
G1	SERVING; server_team=S13	Não executado	-
G2	price_cents=1800; 2 dias	Não executado	-
G3	price_cents=4600; 2 dias	Não executado	-
G4	price_cents=3400; 7 dias	Não executado	-
G5	INVALID_ARGUMENT; INVALID_WEIGHT	Não executado	-

7.3 Outras chamadas do C5

Horário	Chamada	Resposta	Observação
20:31:51	GET /api/v1/products/{sku} sem X-Request-ID	400 MISSING_REQUIRED_HEADER	Conforme o contrato
20:38:33	GET /api/v1/products/KB-100 com X-Request-ID=MN-400	200	SKU digitado no campo do Request ID
20:40:28	POST /api/v1/quotes com KB-100 x 0	422 INVALID_QUANTITY_OR_ITEMS	Conforme o contrato

Horário	Chamada	Resposta	Observação
20:40:42	POST /api/v1/quotes com KB-100 × 1000000000000	422 INVALID_QUANTITY_OR_ITEMS	Conforme o contrato
20:40:52	POST /api/v1/quotes com KB-100 × 1	200	Cotação válida, subtotal 25990 sem desconto

8. Evidências de logs

Os logs completos estão no repositório, em `logs/rest/server.log` e `logs/grpc/server.log`, sem edição.

8.1 REST, Cliente C5

Log do Servidor. As linhas das 20:29 às 20:40 são do C5. A linha das 20:09 é um acesso à raiz do servidor pelo navegador, sem headers.

```
[2026-10-06T20:09:07] protocol=REST server=S13 client=- requestId=- operation=GET:/ input=- status=404 result=INVALID_REQUEST
[2026-10-06T20:29:39] protocol=REST server=S13 client=- requestId=- operation=GET:/ input=- status=404 result=INVALID_REQUEST
[2026-10-06T20:29:58] protocol=REST server=S13 client=- requestId=- operation=GET:/docs input=- status=200 result=OK
[2026-10-06T20:29:59] protocol=REST server=S13 client=- requestId=- operation=GET:/openapi.json input=- status=200 result=OK
[2026-10-06T20:30:29] protocol=REST server=S13 client=- requestId=- operation=GET:/docs input=- status=200 result=OK
[2026-10-06T20:30:44] protocol=REST server=S13 client=- requestId=- operation=GET:/openapi.json input=- status=200 result=OK
[2026-10-06T20:31:51] protocol=REST server=S13 client=S13 requestId=- operation=GET_PRODUCT input=- status=400 result=MISSING_REQUIRED_HEADER
[2026-10-06T20:32:35] protocol=REST server=S13 client=S13 requestId=req-001 operation=GET_PRODUCT input=KB-100 status=200 result=OK
[2026-10-06T20:38:33] protocol=REST server=S13 client=S13 requestId=MN-400 operation=GET_PRODUCT input=KB-100 status=200 result=OK
[2026-10-06T20:39:32] protocol=REST server=S13 client=S13 requestId=req-001 operation=CREATE_QUOTE input=stringx0 status=422 result=INVALID_PRODUCT
[2026-10-06T20:40:28] protocol=REST server=S13 client=S13 requestId=req-001 operation=CREATE_QUOTE input=KB-100x0 status=422 result=INVALID_QUANTITY_OR_ITEMS
[2026-10-06T20:40:42] protocol=REST server=S13 client=S13 requestId=req-001 operation=CREATE_QUOTE input=KB-100x1000000000000000 status=422 result=INVALID_QUANTITY_OR_ITEMS
[2026-10-06T20:40:52] protocol=REST server=S13 client=S13 requestId=req-001 operation=CREATE_QUOTE input=KB-100x1 status=200 result=OK
```

Como o C5 preencheu X-Client-Team com S13, o campo `client` do log não identifica o Cliente, e o mesmo `req-001` aparece em cinco chamadas. A atribuição dessas linhas ao C5 foi confirmada pela equipe durante a janela, pelo horário das chamadas.

Log do Cliente: não há. As chamadas partiram do navegador, pela página `/docs` do servidor, e o C5 não forneceu registros da própria aplicação.

8.2 gRPC, Cliente C5

```
[2026-10-06 19:54:59,963] gRPC server iniciado em 0.0.0.0:50051 | server_team=S13
[2026-10-06 20:13:28,294] gRPC server iniciado em 0.0.0.0:50051 | server_team=S13
[2026-10-06 20:51:02,644] gRPC server iniciado em 0.0.0.0:50051 | server_team=S13
```

O log registra só as três inicializações do servidor (19:54, 20:13 e 20:51), todas com `server_team=S13`. Não há nenhuma chamada `Health` ou `CalculateShipping` no período. Como o servidor registra inclusive as chamadas rejeitadas por falta de `x-client-team`, nenhuma requisição gRPC chegou a ele durante a janela.

Log do Cliente: não há.

9. Falhas observadas e correções

Cada falha foi classificada em uma das quatro categorias do guia: rede ou ambiente, divergência de contrato, regra de negócio ou erro do Cliente. Todas envolvem o Cliente C5. Nenhuma correção alterou o contrato.

Protocolo e teste	Request ID	Sintoma	Categoria	Causa	Encaminhamento
REST e gRPC, todos	-	A aplicação do C5 não executou os testes	Erro do Cliente	Problemas técnicos na aplicação do C5, sem detalhes informados à equipe	O C5 passou a testar o REST pelo Swagger do servidor
REST, R1	-	Erro 400 de header ausente na primeira tentativa	Erro do Cliente	X-Request-ID não preenchido	Reenvio com req-001, que retornou 200
REST, todos	req-001	Log com client=S13 e o mesmo Request ID em cinco chamadas	Erro do Cliente	X-Client-Team preenchido com o código do Servidor e Request ID reaproveitado	Chamadas identificadas pelo horário
REST, consulta de produto	MN-400	Consulta retornou KB-100	Erro do Cliente	SKU digitado no campo X-Request-ID	Nenhum; o servidor respondeu conforme o contrato
REST, R5	req-001	422 INVALID_PRODUCT com o corpo de exemplo	Erro do Cliente	Corpo de exemplo do Swagger enviado sem edição	O C5 ajustou o corpo nas chamadas seguintes
gRPC, G1 a G5	-	Nenhuma chamada recebida	Erro do Cliente	A aplicação do C5 não chegou a chamar o serviço, e o gRPC não tem uma alternativa pelo navegador	Nenhum

Nenhuma falha foi atribuída ao Servidor. Todas as respostas registradas seguem o contrato, e nenhuma correção foi necessária durante a janela.

Ajustes feitos depois da integração. Sem mudar o contrato nem as respostas dos serviços, a equipe organizou o repositório para a entrega: o log do gRPC passou a ser gravado em logs/grpc/server.log, ao lado do log REST; o script testes_grpc.py passou a ler o endereço de GRPC_TARGET em vez de um IP fixo; o código padrão da equipe foi unificado em S00 nos dois servidores; e a cópia do shipping.proto que ficava em grpc-server/proto foi removida, deixando contracts/ como única referência.

10. Comparação prática REST × gRPC

Do ponto de vista de quem implementou os servidores, o gRPC eliminou erros de formato, enquanto o REST exigiu mais validação manual. A integração mostrou o outro lado: o REST foi o único dos dois que o C5 conseguiu testar sem a própria aplicação.

Aspecto	REST	gRPC
Definição do contrato	Documento em texto; cada equipe interpreta paths, campos e JSON	Arquivo .proto compartilhado; os stubs gerados garantem nomes e tipos

Aspecto	REST	gRPC
Validação	Manual: JSON malformatado, tipos errados, campos ausentes	Tipos garantidos pela serialização; restam só as regras de negócio
Tratamento de erros	Status HTTP e corpo JSON padronizado	Status gRPC e descrição em texto
Metadados	Headers HTTP (X-Client-Team, X-Request-ID)	Metadata gRPC (x-client-team)
Testes manuais	Simples com curl, Postman ou navegador; o Swagger do FastAPI permitiu ao C5 testar sem a própria aplicação	Exige um cliente com stubs ou o grpcurl com o arquivo .proto
Formato na rede	JSON legível	Protobuf binário, mais compacto
Configuração inicial	Menor: só o framework web	Maior: geração de código a partir do .proto

O REST exigiu mais código de borda. O FastAPI responde 422 por padrão quando o corpo é inválido, então foi preciso sobrescrever esse tratamento para devolver 400 `INVALID_REQUEST`, usar inteiro estrito em `quantity` (para não aceitar "2") e um middleware para devolver `X-Request-ID` em todas as respostas. No gRPC, os stubs gerados resolveram nomes e tipos, e o trabalho ficou concentrado nas regras de negócio e nos status de erro via `context.abort`.

Na integração, a diferença que mais pesou foi a de teste manual. Quando a aplicação do C5 falhou, o REST continuou testável pelo navegador, sem instalar nada. O gRPC não tinha essa saída: o servidor não habilita `reflection`, então testar exigiria o `.proto` e uma ferramenta como o `grpcurl`, ou um cliente com stubs gerados, e o C5 não chegou a fazer nenhuma chamada.

11. Conclusão

Os servidores REST e gRPC da equipe S13 ficaram ativos durante toda a janela e responderam de acordo com o contrato em todas as chamadas registradas. A integração, porém, ficou limitada pelo lado Cliente: o único Cliente sorteado, o C5, não conseguiu usar a própria aplicação. Pelo Swagger do servidor, o C5 executou R1 e R5 no REST, além de casos de erro, todos com o resultado esperado. R2, R3 e R4 não foram executados, e o gRPC não recebeu chamadas, então G1 a G5 ficaram validados apenas pelos testes internos da equipe.

Por isso a atividade não chegou a demonstrar interoperabilidade entre tecnologias diferentes: as chamadas do C5 saíram de uma página servida pelo próprio servidor, e não de um Cliente escrito por outra equipe. O que ela mostrou foi o peso do contrato e da disciplina do lado Cliente. Os erros registrados (header ausente, código de equipe trocado, Request ID repetido, corpo de exemplo enviado sem edição) foram todos do Cliente, e o servidor respondeu a cada um com o status e o código previstos.

Ficam duas lições para uma próxima rodada. A primeira é que a correlação por Request ID e código de equipe depende do Cliente: com `X-Client-Team=S13` e o mesmo `req-001` em várias chamadas, o log sozinho não distingue o C5 dos testes da própria equipe. Registrar também o IP de origem ajudaria nesse diagnóstico sem alterar o contrato. A segunda é que o gRPC depende mais do Cliente: sem uma aplicação funcionando, a alternativa seria o `grpcurl` com o `.proto` do contrato, algo que vale deixar preparado antes da janela.

Checklist da entrega

Item	Local no repositório	Situação
Código-fonte do Servidor REST e do Servidor gRPC	rest-server/ e grpc-server/	Entregue
README com tecnologia, dependências, portas e comandos de inicialização	README.md e o README de cada servidor	Entregue
Logs completos das chamadas recebidas	logs/rest/server.log e logs/grpc/server.log	Entregue
Relatório técnico em PDF e DOCX	docs/relatorio/	Entregue